Comparative Analysis of Time-Domain and Frequency-Domain Phasor Estimation

Haron Akram Ahmed Mohammed

***Abstract*—As power systems become increasingly complex, Phasor Measurement Units (PMUs) are essential for real-time monitoring. Accurate measurements are therefore necessary for timely and reliable system operation. This thesis refines the digital signal processing of an open-source PMU built on a Raspberry Pi and BeagleBone Black. It compares two phasor estimation methods; a time-domain Least Squares Method (LSM) and a frequency-domain Enhanced Interpolated Discrete Fourier Transform (e-IpDFT). To satisfy the accuracy requirements of the IEEE C37.118 P-Class standards, the mathematical framework of the e-IpDFT was utilized, while the algorithm was programmed in C++ and linked to Python to ensure real-time performance. Both algorithms were tested via MATLAB simulations and live hardware. Results show that while the LSM performs well at the power system's nominal frequency, it assumes the grid frequency is always exactly 50 Hz and fails during frequency deviations, reaching a Total Vector Error (TVE) near 32%. In contrast, the e-IpDFT adapts to the actual grid frequency, offering a more robust solution that keeps TVE, Frequency Error (FE), and Rate-of-Change-of-Frequency Error (RFE) well below the standard IEEE limits. Live tests confirmed that the e-IpDFT is a reliable, computationally efficient refinement to the LSM, making the PMU better equipped to provide accurate estimations during grid fluctuations and disturbances.**

***Sammanfattning*—I takt med att elnät blir alltmer komplexa är fasmätningsenheter (PMU) avgörande för realtidsövervakning. Noggranna mätningar är därför nödvändiga för en snabb och pålitlig systemdrift. Denna avhandling förfinar den digitala signalbehandlingen av en öppen källkods-PMU byggd på en Raspberry Pi och BeagleBone Black. Den jämför två metoder för fasuppskattning; en minstakvadratmetod (LSM) i tidsdomänen och en förbättrad interpolerad diskret Fouriertransform (e-IpDFT) i frekvensdomänen. För att uppfylla de strikta noggrannhetskraven i IEEE C37.118 P-klass-standarden användes det matematiska ramverket för e-IpDFT, medan algoritmen programmerades i C++ och länkades till Python för att säkerställa prestanda i realtid. Båda algoritmerna testades via MATLAB-simuleringar och live-hårdvara. Resultaten visar att även om LSM presterar väl vid elnätets nominella frekvens, antar den att nätfrekvensen alltid är exakt 50 Hz och misslyckas därmed vid frekvensavvikelser, och når ett totalt vektorfel (TVE) nära 32 %. Däremot anpassar sig e-IpDFT till den faktiska nätfrekvensen, vilket erbjuder en mer robust lösning som håller TVE, frekvensfel (FE) och derivatafel (RFE) långt under de fastställda IEEE-gränserna. Livetester bekräftade att e-IpDFT är en pålitlig och beräkningseffektiv förfining av LSM, vilket gör PMU:n bättre rustad att ge korrekta uppskattningar under fluktuationer och störningar i elnätet.**

***Index Terms*—Phasor Measurement Unit (PMU), Synchrophasor Estimation, Enhanced Interpolated DFT (e-IpDFT), Least Squares Method (LSM), IEEE C37.118, OpenPMU, Digital Signal Processing.**

Supervisor: Valgerður Jónsdóttir

TRITA number: TRITA-EECS-EX-2026:109

I. Introduction

Modern power systems are becoming increasingly complex due to the integration of renewable energy sources. Because solar and wind power reduce the natural mechanical inertia of the grid, frequency fluctuations happen much faster [1]. This requires fast and accurate monitoring systems to enable rapid response to disturbances and support control actions that restore system stability, such as frequency or voltage control. Traditional Supervisory Control and Data Acquisition (SCADA) systems usually provide measurements every 2 to 4 seconds, which is too slow for the control system to react, whether in local protection and automation or for system-wide control in centralized control rooms [2].

To address this limitation, Phasor Measurement Units (PMUs) are used. PMUs sample analog waveforms at high frequencies and use global positioning systems (GPS) to give every measurement a precise microsecond timestamp [3]. By collecting these synchronized measurements across multiple network locations, grid operators are better equipped to see the true real-time state of the power grid.

A. Project Scope and Objectives

This thesis builds on a locally developed PMU architecture, which was originally adapted from the open-source OpenPMU framework [4]. Within this framework, all required modules for setting up and operating a PMU were provided. One of those modules is the phasor estimation algorithm, which handles the estimation of frequency, amplitude, and phase angle of the waveforms. For further development of the local PMU, this thesis focuses on the refinement of this phasor algorithm.

The existing setup uses a Least Squares Method (LSM) to estimate the parameters of the signal in the time domain. This project introduces a frequency-domain approach based on the Discrete Fourier Transform (DFT). Since the IEEE C37.118 standard sets strict limits on measurement errors, several variations of the DFT approach were considered to find a robust algorithm capable of meeting the requirements of [5]. Specifically, the algorithms are measured against the IEEE C37.118 P-Class limits[1]. Compliance is evaluated using the limits for Total Vector Error (TVE), Frequency Error (FE),

[1]The IEEE C37.118 standard defines two performance classes: P-Class (Protection), intended for equipment requiring fast response times during dynamic grid faults, and M-Class (Measurement), intended for high-accuracy applications where reporting speed is less critical.

and Rate-of-Change-of-Frequency Error (RFE) across both simulated and physical testing environments.

The final objective is to benchmark the LSM against the chosen DFT implementation in a real-time embedded environment. However, because this thesis was an independent effort, bridging the gap between math and real-world hardware required hands-on work. As a result, the project also involved elements of device development to troubleshoot and deploy the code on the physical hardware. Ultimately, the time and scope of the project allowed for setting up and refining the algorithms for a single PMU, located at KTH in Stockholm.

II. Background

This section provides the background for the thesis. First, it introduces synchrophasors, then describes the existing PMU hardware built according to the OpenPMU framework, and finally describes the controlled hardware evaluation setup.

A. Synchrophasor Fundamentals

In the context of power systems, a continuous voltage or current waveform in the time domain can be mathematically expressed as:

$$x(t) = A_m \cos(2\pi f t + \theta) + \varepsilon(t) \tag{1}$$

where A_m is the peak amplitude, f is the frequency, θ is the phase angle, and $\varepsilon(t)$ represents any harmonics, interharmonics, or noise present in the signal. It is important to note that this actual frequency f may differ from the power system's nominal frequency of 50 or 60 Hz.

A synchrophasor is the complex representation of the fundamental frequency component of the waveform. By ignoring the noise $\varepsilon(t)$ and mapping the pure sine wave onto the complex plane, it is written as $X(t) = \frac{A_m}{\sqrt{2}} e^{j\phi}$, where $\phi = 2\pi f t + \theta$ [3], as visualized in Figure 1.

Calculating the phase angle requires a global time reference because the angle changes constantly over time. For that, PMUs use GPS time signals, that allow Phasor Data Concentrators (PDCs) to align incoming data packets from different PMUs and create a real-time snapshot of the grid.

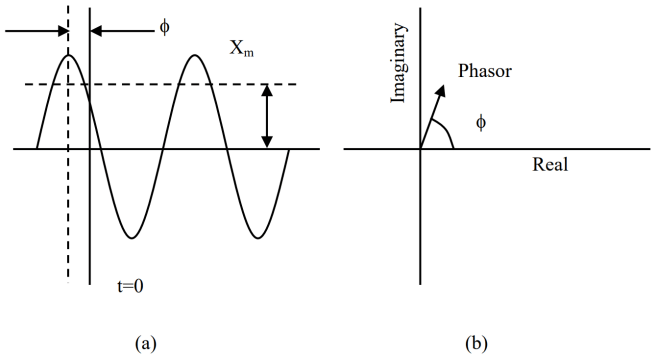


Fig. 1. A power system signal represented in the time domain as a continuous wave (a) and mapped onto the complex plane as a static synchrophasor (b). The magnitude X_m represents the Root Mean Square (RMS) value, while the phase angle ϕ is determined by the $t = 0$ reference. Adapted from [3, p. 6].

B. The OpenPMU Framework and Previous Work

As mentioned in Section I-A, this project builds on an existing project at KTH, detailed in [6], that used the OpenPMU framework to build PMU prototypes. They used BeagleBone Black (BBB) microcomputers and custom analog-to-digital converter (ADC) boards to create a pipeline that digitizes AC waveforms and sends User Datagram Protocol (UDP) data to a server over a Virtual Private Network (VPN).

The hardware and network they built worked well, but the signal processing relied entirely on the baseline Least Squares Method, provided by the OpenPMU framework. As previously noted, the frequency of the power system's signal is not guaranteed to remain at the nominal 50 Hz. In a power grid, synchronous generators all produce power at the same rotational speed, which establishes the system's frequency. Under ideal conditions, this is equal to the nominal frequency. However, any instantaneous mismatch between power generation and consumer demand causes this rotational speed, and thus the grid frequency, to deviate.

The baseline LSM assumes that the waveform in question is exactly at the nominal frequency. Because this is not always the case, the LSM struggles to provide accurate measurements at off-nominal frequencies. To develop a more robust algorithm that can accurately estimate the phasor at off-nominal frequencies (such as 48 Hz or 52 Hz), a Fourier-based approach is explored as an alternative.

C. Controlled Hardware Evaluation

To accurately compare the performance of the existing LSM algorithm against the frequency-domain algorithm, both algorithms were developed simultaneously on a single PMU at KTH.

The setup provided a major analytical benefit. By feeding the exact same raw input signal to both mathematical engines inside one device, the physical grid conditions became a controlled variable. Any difference in the output data could be directly tied to the mathematical processing, completely removing geographic, network, or hardware variations from the equation.

Running two heavy algorithms simultaneously on one microcomputer initially raised concerns about CPU bottlenecks. As a result, computational efficiency and code optimization became a necessary part of the testing environment.

III. Phasor Estimation and PMU Performance Metrics

This section details the math behind the phasor estimation algorithms tested in this thesis. It covers the time-domain LSM approach, the progression of frequency-domain estimators, and the standardized metrics and steady-state tests used to evaluate estimation accuracy.

A. Time-Domain Estimation: Least Squares Method

The LSM algorithm uses a Least Squares curve-fitting method. It assumes the grid signal is a pure sine wave fixed

exactly at the nominal power system frequency (50 Hz), with a possible DC offset.

To avoid solving complex non-linear equations in real-time, the algorithm linearizes the signal model. Adapting the notation from Equation (1), the model assumes the noise component $\varepsilon(t)$ is simply a DC offset C, yielding $y(t) = A_m \cos(\omega_0 t + \theta) + C$. Using trigonometric identities, this expands into:

$$y(t) = b_1 \cos(\omega_0 t) + b_2 \sin(\omega_0 t) + b_3 \tag{2}$$

where $\omega_0 = 2\pi f_0$ is the nominal angular frequency (50 Hz). For a window of N samples, a system of linear equations is set up as a matrix:

$$\mathbf{C} = \mathbf{XB} \tag{3}$$

Here, $\mathbf{X}$ is the observation matrix with the sine and cosine templates, matrix $\mathbf{C}$ is formed using the discrete data samples (the sampled voltages) in this least-squares regression context, and $\mathbf{B} = [b_1, b_2, b_3]^T$ is the parameter vector. The system is solved using the Moore-Penrose pseudo-inverse [7]:

$$\mathbf{B} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{C} \tag{4}$$

From $\mathbf{B}$, the peak amplitude A_m and phase angle θ are extracted. The frequency is then calculated by tracking how fast the phase angle changes over time.

B. Frequency-Domain Estimation: Fourier-Based Methods

Unlike the LSM approach, which directly fits the signal to a fixed 50 Hz sinusoidal template, frequency-domain methods estimate the signal spectrum using Fourier-based techniques. Finding an accurate Fourier-based algorithm required looking at a progression of different methods.

1) Standard Discrete Fourier Transform (DFT): The standard DFT method transforms a time-domain signal into its discrete frequency bins in the frequency domain. Since the DFT can only process a finite number of samples at a time, the signal is analyzed over a limited time interval, referred to as a window. The choice of the window length T directly affects the frequency resolution; a shorter window results in wider spacing between frequency bins, whereas a longer window provides finer frequency resolution, thereby improving the estimation. The spacing between the frequency bins is given by $\Delta f = 1/T$.

The standard DFT applies the rectangular window, meaning it simply cuts the signal off at the start and end. If the actual grid frequency deviates from the nominal frequency, the signal may no longer align exactly with one of the discrete frequency bins. In this case, the signal energy spreads across multiple frequency bins rather than remaining concentrated at a single frequency. This effect is referred to as spectral leakage, which creates sidelobes in the frequency spectrum that can introduce errors in the phasor estimation.

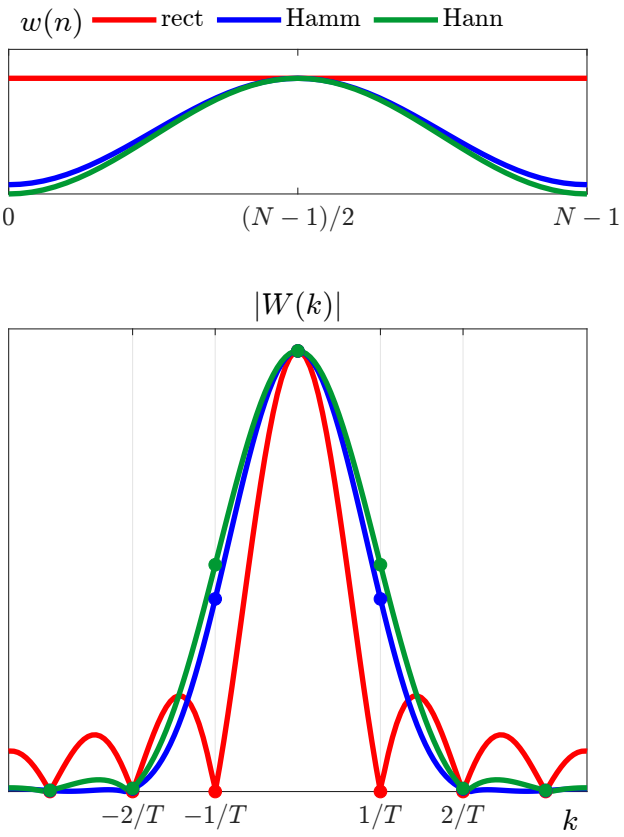


Fig. 2. Time- (top) and frequency-domain (bottom) representations of three different window functions: rectangular (red), Hamming (blue), and Hanning (green). Adapted from [9].

2) Interpolated DFT (IpDFT): To reduce spectral leakage, the Interpolated DFT method uses a tapering window and interpolation techniques to better estimate off-nominal frequencies [8], [9].

Unlike the rectangular window, which has a coherent gain of 1.0 but creates sharp edges, tapering windows like the Hamming and Hanning windows are used:

$$w_{hamm}[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N}\right) \tag{5}$$

$$w_{hann}[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N}\right) \tag{6}$$

where N is the total number of samples in the windowed signal. As shown in Figure 2, while the Hamming window has a gain of 0.54, it does not reach exactly zero at the edges. The Hanning window has a gain of 0.5 and forces the amplitude to exactly zero at both ends. This smooth transition makes the sidelobes drop off much faster, keeping the spectral leakage close to the main frequency.

After applying the window, the IpDFT method applies interpolation to the two highest frequency bins in the spectrum, where most of the signal energy is typically concentrated. By comparing the magnitudes of these two bins, it estimates a fractional bin offset, δ. This offset acts as a fine-tuning adjustment, locating the exact grid frequency even if it falls between two discrete frequency bins. As proven by Grandke in [8], when a Hanning window is used, the phase error from

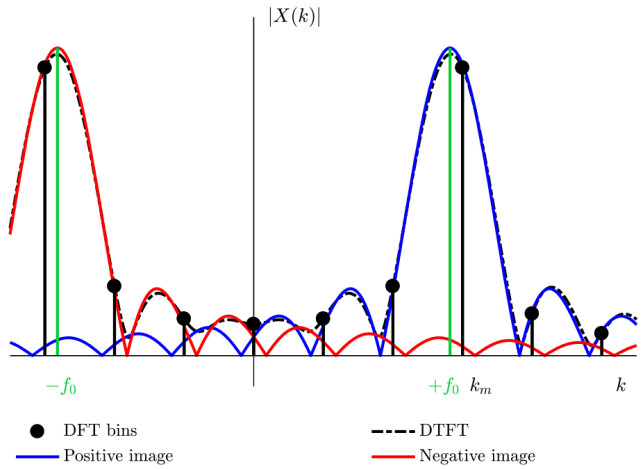


Fig. 3. Frequency spectrum of a windowed real-valued sinusoidal signal. The continuous curve illustrates the true underlying spectrum (DTFT), while the distinct markers represent the specific frequency bins calculated by the algorithm (DFT). By displaying both, the graph clearly demonstrates how the continuous spectral leakage (the long tails) from the negative frequency image extends across the zero-axis and artificially interferes with the positive fundamental frequency bins. Adapted from [9].

this offset is linear. The true phase angle is recovered using Grandke's correction:

$$\Delta\varphi = \pi \cdot \delta \tag{7}$$

3) Enhanced-Interpolated DFT (e-IpDFT): While the IpDFT mitigates spectral leakage from nearby bins, estimation errors can still be affected by the negative frequency spectrum and its interference on the positive spectrum. A real-valued sinusoidal signal contains both a positive frequency component ($+f_0$) and a negative frequency component ($-f_0$).

For a 50 Hz power system signal, the positive and negative fundamental frequencies are relatively close to each other in the frequency spectrum. Consequently, the continuous spectral leakage from the negative frequency image extends across the zero-axis and artificially interferes with the positive fundamental frequency bins, as illustrated in Figure 3. This interference occurs regardless of window length, though it becomes even more pronounced when shorter observation windows are used.

Based on the method proposed in [9], the Enhanced-IpDFT aims to fix this by iteratively compensating for the spectral leakage from the negative frequency spectrum. At each iteration, the method estimates the parameters of the negative frequency spectrum, subtracts its interference from the positive-frequency bins, and recalculates the fractional bin offset to refine the phasor estimation. This process is repeated until a more accurate measurement is obtained.

C. IEEE C37.118 Performance Metrics and Testing

The algorithms were evaluated using the following three standardized metrics defined in the IEEE C37.118.1 synchrophasor standard [5, Sec. 5.3]:

- **Total Vector Error (TVE):** The difference between the theoretical true complex phasor ($X_r + jX_i$) and the estimated complex phasor ($\hat{X}_r + j\hat{X}_i$).

$$TVE = \sqrt{\frac{(\hat{X}_r - X_r)^2 + (\hat{X}_i - X_i)^2}{X_r^2 + X_i^2}} \times 100\% \tag{8}$$

- **Frequency Error (FE):** The absolute difference between the true grid frequency (f_{true}) and the estimated frequency (f_{est}).

$$FE = |f_{est} - f_{true}| \tag{9}$$

- **Rate-of-Change-of-Frequency Error (RFE):** The absolute difference between the true Rate of Change of Frequency (ROCOF) ($ROCOF_{true}$) and the estimated ROCOF ($ROCOF_{est}$).

$$RFE = |ROCOF_{est} - ROCOF_{true}| \tag{10}$$

The phasor estimation methods described earlier, both the LSM and the DFT-based algorithms, were evaluated in a controlled MATLAB environment. To verify compliance before physical deployment, they were tested against the specific P-Class limits shown in Table I.

TABLE I
IEEE C37.118 P-CLASS STEADY-STATE ERROR LIMITS

Compliance Class	Max TVE	Max FE	Max RFE
P-Class (Protection)	$\leq 1.0\%$	≤ 0.005 Hz	≤ 0.4 Hz/s

Three steady-state tests were conducted using synthetic grid waveforms in MATLAB to evaluate the performance of the algorithms in terms of TVE, FE, and RFE:

1) **Signal Frequency Sweep:** The grid frequency was shifted in 0.1 Hz steps from 48.0 Hz to 52.0 Hz to evaluate performance at off-nominal frequencies.
2) **Signal Magnitude Step:** The voltage amplitude was changed in 10% steps (from 0.1 p.u. to 2.0 p.u.) to assess robustness to variations in signal magnitude.
3) **Harmonic Distortion:** Single harmonics (2nd to 50th) were injected at 1% of the fundamental amplitude to test the algorithm's ability to reject interference.

IV. SYSTEM ARCHITECTURE

This section outlines the physical and digital setup of the PMU. It defines the hardware components, the software pipeline, and the visualization tools used for the experiments.

A. Hardware Infrastructure and Secure Networking

The PMU setup consists of custom analog circuitry and two microcomputers. The three-phase grid signal enters an analog input board, which scales down the high voltage. This scaled signal goes into a BBB equipped with an ADC, shown in Figure 4.

The BBB acts as a high-speed digitizer. It samples the waveform at 10,000 Hz and sends this raw data over Ethernet using UDP. A Raspberry Pi receives the UDP stream and runs the signal processing algorithms. To keep the data secure when sending the final measurements over the internet, the Raspberry Pi uses an OpenVPN server. This creates a private,

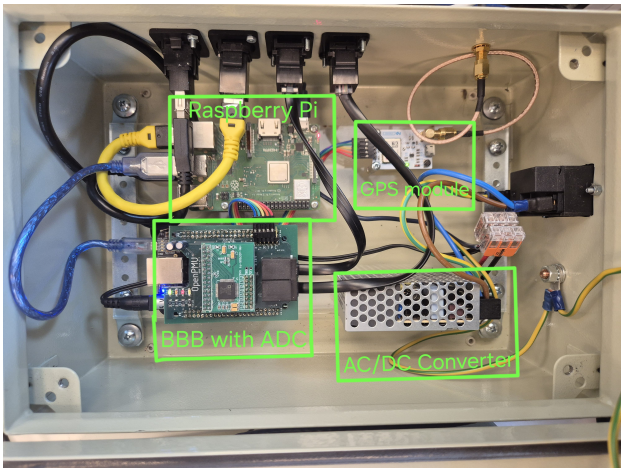


Fig. 4. The physical PMU deployed at KTH, highlighting the key components: the BeagleBone Black with the custom analog-to-digital (ADC) expansion board, the GPS module, the AC/DC converter, and the Raspberry Pi.

encrypted tunnel between the local PMU and the remote data concentrator, preventing unauthorized access or tampering with the grid data.

B. Modular Software Pipeline

The OpenPMU framework runs as a modular pipeline made of several Python scripts. These scripts communicate using internal Multicast IP routing (e.g., 239.16.1.101). This setup allowed new mathematical algorithms to be inserted without affecting or breaking the network logic, as illustrated by the data flow diagram in Figure 5. The complete Python and C++ source code for these modules, along with the 3D CAD models of the physical PMU enclosures and the MATLAB implementation, are all available in the project's GitHub repository [10].

To achieve a direct comparison on a single device, the internal pipeline was duplicated. A single `MulticastSV` node captures the raw data and broadcasts it to two separate estimation modules running side-by-side. Their outputs are then handled by dedicated, duplicated logging and communication scripts.

The core software modules include:

- **MulticastSV:** The entry point for the raw data. It catches the incoming UDP sampled values (SV) from the BBB and retransmits them to the Raspberry Pi's internal multicast group.
- **PhasorEst (01 & 02):** The core phasor estimation engines. Two instances run simultaneously: one applies the time-domain LSM, and the other applies the frequency-domain e-IpDFT via the compiled C++ wrapper.
- **Telecom (01 & 02):** Packages the phasor data according to the IEEE C37.118-2011 protocol and prepares it for transmission over distinct network ports (Port 4711 and 4712).
- **CSVlogger (01 & 02):** Locally logs the phasor estimations into CSV files on a mounted USB drive as a hardware-level backup in case the network drops.

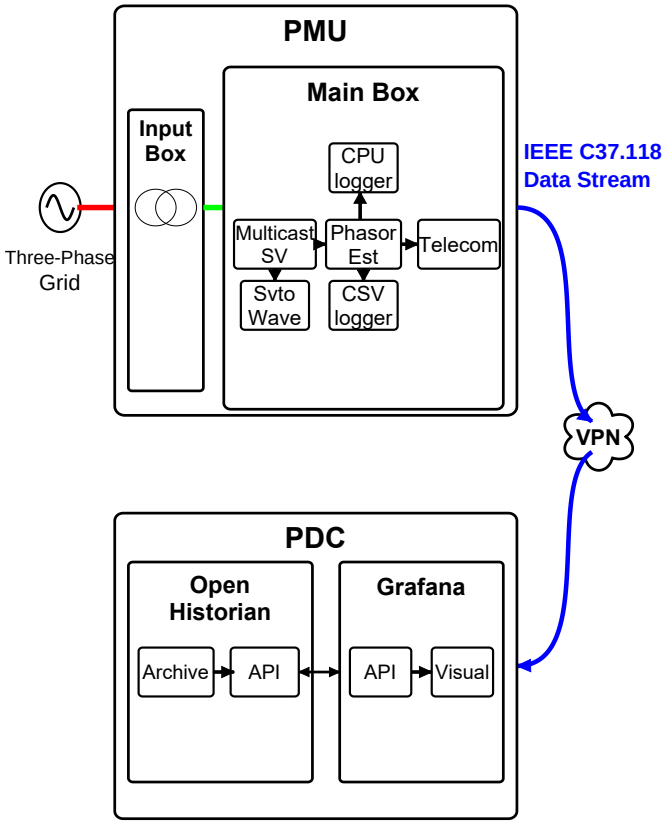


Fig. 5. Flow diagram of the internal OpenPMU modular software architecture, illustrating the data path from raw UDP sampling to IEEE C37.118 transmission. Adapted from [6].

- **SvtoWave:** A debugging tool that saves the raw 10 kHz sampled values as `.flac` audio files. To save disk space, it only records the first 5 minutes of every hour.
- **CPUlogger:** A custom module made for this thesis to record the CPU usage of both algorithms. It was set to poll every 5 seconds so the measuring process itself did not slow down the Raspberry Pi.

C. Data Concentration and Visualization

After the Telecom module packages the data, it is sent over the VPN to the Phasor Data Concentrator (PDC). In this setup, the PDC consists of two connected systems.

The first is openHistorian, which is a time-series database. It receives the IEEE C37.118 data, aligns the measurements using their GPS timestamps, and archives them. The second is Grafana, an open-source analytics platform. Grafana uses an Application Programming Interface (API) to pull the archived data from openHistorian and display it on live graphical dashboards. This setup allowed for real-time visual comparison of the output from the different algorithms: the frequency, amplitude, and phase estimated from both the LSM algorithm and the DFT-based algorithm.

The PDC was initially deployed on a local workstation. However, this configuration proved inadequate for continuous monitoring, as host power state transitions (e.g., sleep mode) terminate the network connection and halt data transmission. To guarantee uninterrupted grid observation, the PMU stream

is securely routed to a high-availability Phasor Data Concentrator (PDC) hosted at Lund University, acting as a central node for the Nordic University network [11].

Exposing the PMU to an external wide-area network required strict security configurations to prevent unauthorized access and ensure data integrity. The PMU transmits the measurements using the IEEE C37.118-2011 protocol, configured in Commanded Mode over Transmission Control Protocol (TCP). To secure the connection, the Uncomplicated Firewall (UFW) on the Raspberry Pi was configured with strict IP whitelisting. The firewall exclusively permits the Lund University concentrator's specific IP address to access the data stream on a dedicated TCP port.

Meanwhile, to protect ongoing research and maintain cryptographic security, the experimental frequency-domain e-IpDFT stream and all administrative SSH access are completely blocked from the public internet. These secondary ports are strictly bound to the internal VPN subnet. This dual-layer routing ensures that the external university receives a secure, uninterrupted data feed, while local researchers retain secure, isolated access to monitor both algorithms simultaneously.

Furthermore, to provide hardware-level redundancy in the event of a wide-area network outage, the dual `CSVlogger` modules archive all measurements directly to a local USB flash drive. Because continuous, high-frequency data logging can quickly exhaust storage capacity, leading to filesystem crashes and severe NAND flash wear, an automated 30-day rolling retention policy was programmed into all local loggers (`CSVlogger`, `SvtoWave`, and `CPUlogger`). This logic ensures that the Nordic University server receives an uninterrupted, infinite stream of data, while the PMU manages its own local storage, guaranteeing data recovery for recent events without compromising the long-term physical health of the embedded hardware.

V. Experimental Setup

This section describes the steps taken to test and deploy the algorithms. The process was split into two phases: a MATLAB simulation to evaluate the IEEE compliance metrics defined in Section III-C, and a cross-language translation to run the algorithms on the embedded hardware.

A. MATLAB Baseline Simulation

To run the steady-state tests, a custom waveform generator (`generate_signal.m`) was written in MATLAB. It created a digital grid signal sampled at 10,000 Hz and allowed for frequency deviations and harmonic injections. To simulate the real BeagleBone hardware, Additive White Gaussian Noise (AWGN) was added to the signal with a Signal-to-Noise Ratio (SNR) of 80 dB.

A processing script (`PMU.m`) was created to feed this simulated signal into the different phasor estimation algorithms: LSM, standard DFT, IpDFT, and e-IpDFT. The goal of this phase was to record the TVE, FE, and RFE outputs during the sweeps to determine which Fourier-based method was highly-compliant with the IEEE thresholds and advance to the hardware phase. This was managed by a central execution script (`PhasorEstimation_RunScript.m`), which iteratively fed processed simulated signals using the (`PMU.m`) library.

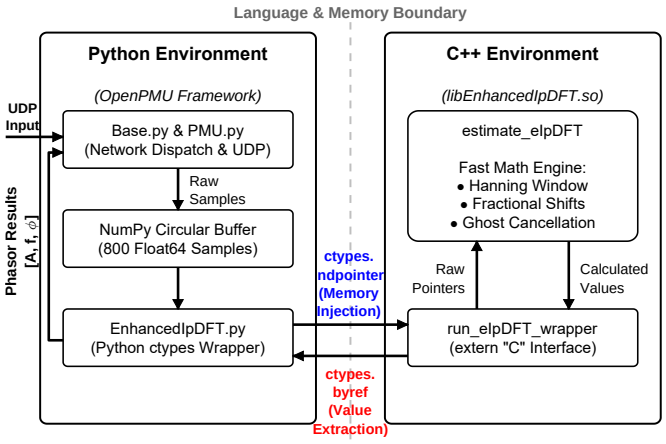


Fig. 6. The cross-language software architecture used to implement the e-IpDFT algorithm on the Raspberry Pi. The system uses the ctypes library to bridge the Python-based OpenPMU network framework with a compiled C++ math engine. By passing raw memory addresses (`ndpointer`) rather than copying data arrays point-by-point, the system bypasses Python's garbage collector to achieve stable, real-time processing at 10 kHz.

B. Embedded Cross-Language Implementation

Once the algorithms were tested in MATLAB, the highly-compliant frequency-domain method had to be translated for the Raspberry Pi.

The OpenPMU architecture is written in Python. However, Python is generally too slow to run heavy, sample-by-sample signal processing calculations at 10,000 Hz. To solve this, a cross-language approach was used. As illustrated in Figure 6, the real-time processing bottleneck was eliminated by splitting the architecture across a strict language and memory boundary.

On the Python side, the OpenPMU framework manages the incoming UDP data stream and buffers the raw 10 kHz voltage samples into a pre-allocated NumPy array. Because copying thousands of data points into a different language environment every fraction of a second would be computationally expensive, the Python wrapper uses the `ctypes` library to pass the physical memory address of the buffer directly to the C++ shared library (`libEnhancedIpDFT.so`).

This allows the C++ engine to instantly read the data and perform the heavy calculations, such as applying the Hanning window, calculating fractional bin offsets, and executing the iterative negative image compensation. Finally, the calculated phasor values (amplitude, frequency, and phase) are pulled back into Python using reference pointers (`byref`) and dispatched to the data concentrator. This approach successfully isolated the high-speed math from Python's standard garbage collection, ensuring the Raspberry Pi could maintain real-time performance without dropping frames.

C. Filtering Parameters

During the translation to the Raspberry Pi, post-processing filters were applied to the raw frequency outputs. Raw frequency calculations can be volatile because hardware white noise affects the phase derivative calculations.

To smooth the data, an Exponential Moving Average (EMA) filter was applied to the frequency and ROCOF estimations. For the live hardware benchmarking, a filter weight of $\alpha = 0.05$ was set for both the LSM and the new C++ e-IpDFT engine.

D. Real-Time Benchmarking Setup

In the final testing phase, both the LSM and the compiled e-IpDFT algorithm were placed on the physical PMU at KTH. The BBB sent the same live 10 kHz voltage samples to both modules at the same time.

The `CPUlogger` recorded the processing strain of each algorithm on the Raspberry Pi. The final data streams were then sent over OpenVPN to the PDC, where Grafana dashboards were used to observe how the two algorithms handled real hardware noise.

VI. Results

This section presents the data collected during both the MATLAB simulation phase and the live embedded hardware deployment. The results strictly report the observed performance, stability, and computational load of the evaluated algorithms, specifically comparing the LSM against the finalized e-IpDFT.

A. Simulation Results: IEEE Compliance and Disturbances

The first phase of results details the compliance of the algorithms against the IEEE C37.118 P-Class standard. Table II summarizes the maximum errors recorded for both the LSM and the e-IpDFT algorithms during the steady-state frequency sweep, amplitude step, and harmonic distortion tests.

To visualize the performance, the error profiles are plotted in Figure 7a (Frequency Sweep), Figure 7b (Magnitude Sweep), and Figure 7c (Harmonic Distortion). A logarithmic scale is used for the y-axis in these figures to clearly display the large difference in error magnitudes between the two estimation methods. The graphs show how the e-IpDFT algorithm remains highly stable and well below the IEEE limits. In contrast, the LSM not only shows large errors when the grid frequency moves away from 50 Hz, but it also violates the IEEE limits in all steady-state tests, even when the fundamental frequency of the signal is exactly at the nominal 50 Hz.

Beyond steady-state testing, the algorithms were subject to dynamic grid disturbances, specifically a +0.5 Hz frequency step and a +10 degree phase jump. It is important to note that a pure, instantaneous frequency step is physically impossible in a real-world power grid due to the mechanical inertia of synchronous generators. However, this theoretical test is included because it provides excellent insight into the algorithmic limitations of the tracking methods. Figure 8 illustrates the transient response of both algorithms during these sudden offsets. The data shows heavy oscillations in the LSM output during the recovery period, while the e-IpDFT transitions smoothly back to a stable state.

TABLE II
Maximum Errors Recorded During Steady-State Testing

Algorithm	Max TVE (%)	Max FE (Hz)	Max RFE (Hz/s)
IEEE Limit	1.000	0.0050	0.4000
Frequency Sweep (48.0–52.0 Hz)			
LSM	31.985	0.4189	23.6833
e-IpDFT	0.016	0.0002	0.0094
Amplitude Sweep (0.1–2.0 p.u.)			
LSM	3.129	0.2501	13.1421
e-IpDFT	0.009	0.0002	0.0080
Harmonic Distortion (2nd–50th)			
LSM	3.127	0.2501	13.1409
e-IpDFT	0.040	0.0014	0.0300

B. Embedded CPU Performance

To determine if running two heavy mathematical engines simultaneously on a single Raspberry Pi caused hardware bottlenecks, the `CPUlogger` module recorded the processor's utilization. To ensure statistical reliability, data was collected over a continuous seven-day period, specifically from April 21, 2026, at 16:00:00 UTC to April 27, 2026, at 16:00:00 UTC.

The processor load stayed constant throughout the entire week without any major spikes or drops. Therefore, instead of showing a flat 7-day graph, the results are presented as an overall average in Table III. The data reveals a difference in the baseline CPU strain required to execute the time-domain matrix calculations of the Python-based LSM compared to the compiled C++ frequency-domain framework of the e-IpDFT.

C. Real-Time Grid Measurements

The final set of results was extracted from the live Grafana dashboards connected to the Phasor Data Concentrator. To ensure a fair comparison, the following dashboard captures were all recorded over the exact same one-minute observation window: April 28, 2026, from 18:32:00 UTC to 18:33:00 UTC.

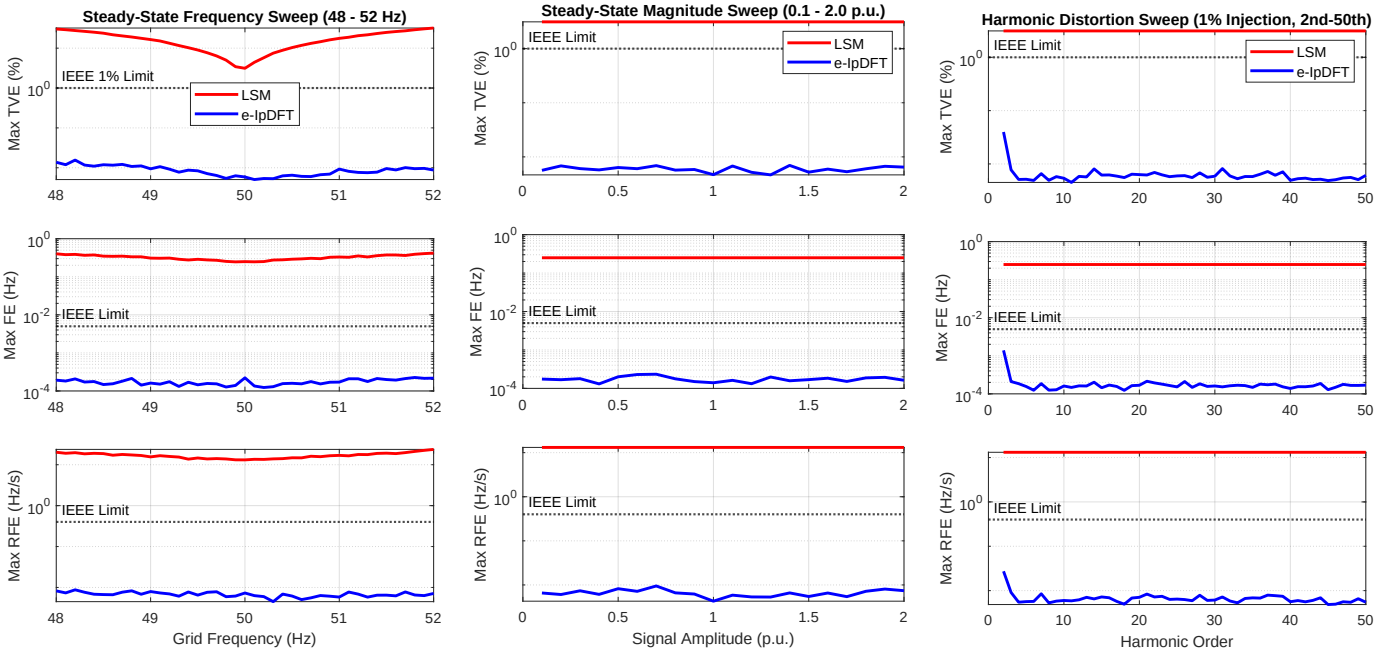
Frequency Tracking against Svenska Kraftnät: To verify the accuracy of the PMU tracking real grid dynamics, official grid frequency data published by Svenska Kraftnät [12] was plotted for the observation window in Figure 9a. The Grafana outputs of both PMU algorithms, shown in Figure 9b, successfully tracked the major trends of this national grid data. Applying the $\alpha = 0.05$ filter effectively reduced hardware noise, creating smooth and readable frequency traces.

Voltage Magnitude Accuracy: When comparing the estimated voltage amplitudes on Phase A, shown in Figure 10, a clear difference between the two algorithms was observed. The Grafana capture shows the e-IpDFT displaying a slightly tighter and more stable voltage trace than the LSM. This shows a difference in how time-domain and frequency-domain algorithms handle real-world hardware noise.

Phase Delay Observations (Device Diagnostics): Figure 11 shows the estimated phase angles from both algorithms

TABLE III
Average Embedded CPU Utilization (7-Day Period)

Algorithm	Avg. CPU Usage (%)
LSM (Python)	14.86
e-IpDFT (C++)	13.34



(a) Frequency Sweep (48–52 Hz). (b) Magnitude Sweep (0.1–2.0 p.u.). (c) Harmonic Distortion (2nd–50th).

Fig. 7. Steady-state compliance tests presented with a logarithmic y-axis. The e-IpDFT remains well within IEEE limits across all tests. In contrast, the LSM exceeds the limits substantially under frequency deviations and maintains an offset above the limit under magnitude and harmonic variations.

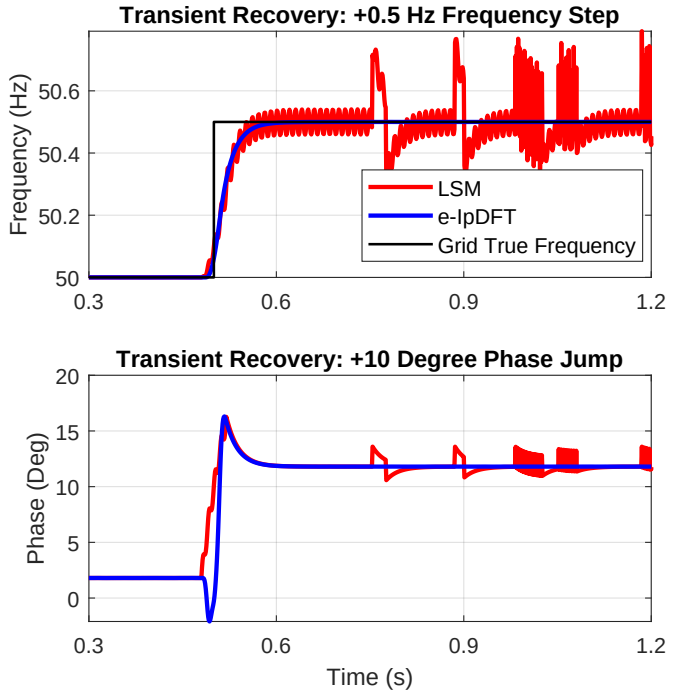


Fig. 8. Simulated transient recovery of LSM and e-IpDFT during an injected +0.5 Hz frequency step (top) and a +10 degree phase jump (bottom). Visible oscillations occur in the LSM output before settling. While a pure frequency step is physically impossible in a real grid, the test is retained to clearly illustrate the algorithmic limitations of the LSM.

side-by-side in Grafana. Even though both algorithms analyzed the exact same data stream on the exact same microcomputer, one algorithm consistently trailed the other by about 2 seconds. The standard mathematical phase wrapping (the vertical drop from $+180^\circ$ to -180°) also happened asynchronously. The underlying cause of this temporal offset is examined further in the discussion section.

Three-Phase Consistency: To verify that the new e-IpDFT wrapper works across a full three-phase system, the voltage magnitudes of all three phases (Phase A, B, and C) were captured in Grafana. They are shown in Figure 12. The algorithm processed the multi-dimensional arrays without memory errors and maintained consistent proportional scaling across all phases during the live test.

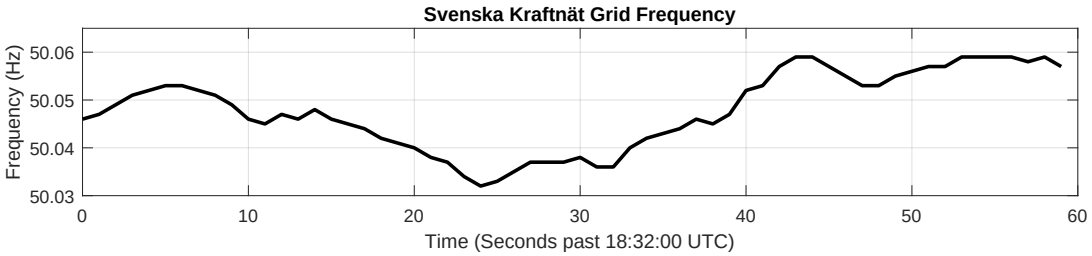
VII. Discussion

The results from this project show a clear difference between mathematical simulations and real-world hardware deployment. This section analyzes the algorithm performance, explains the hardware anomalies, and discusses the sources of error in the implementation.

A. Algorithm Comparison: Time-Domain vs. Frequency-Domain

The MATLAB simulations clearly demonstrated that the Least Squares Method (LSM) fails to meet IEEE C37.118 limits under both nominal and off-nominal grid conditions. As seen in Figure 7a, when the frequency moved toward 48 Hz or 52 Hz, the LSM reached a Total Vector Error (TVE) of nearly 32%, a Frequency Error (FE) of 0.4189 Hz, and a Rate-of-Change-of-Frequency Error (RFE) of 23.68 Hz/s. These values are drastically above their respective P-Class limits.

This happens because the LSM uses a fixed 50 Hz observation matrix. When the grid frequency changes, the incoming signal no longer fits this matrix. This mismatch creates a 100 Hz ripple in the calculations, which causes the massive



(a) Official grid frequency from Svenska Kraftnät.

(b) Local PMU frequency at KTH (LSM in yellow, e-IpDFT in green).

Fig. 9. Frequency comparison over the same observation window (18:32:00–18:33:00 UTC). The official reference (a) and the local PMU estimates (b) follow the same overall trend.

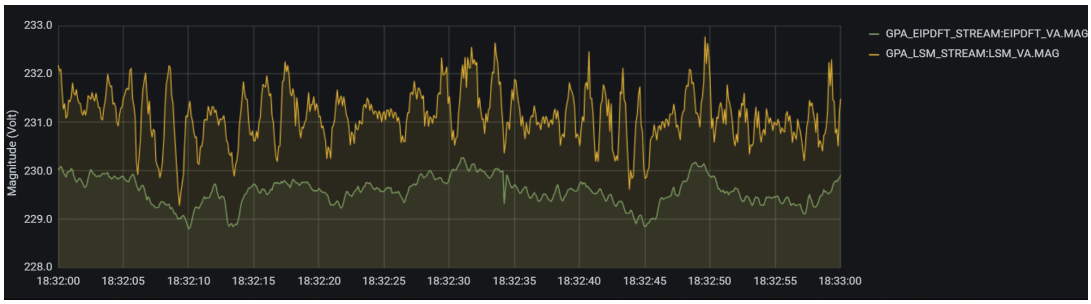


Fig. 10. Live voltage magnitude (V_A) comparison over the observation window. The data reveals a slight variance in amplitude stability between the time-domain (LSM in yellow) and frequency-domain (e-IpDFT in green) estimations.

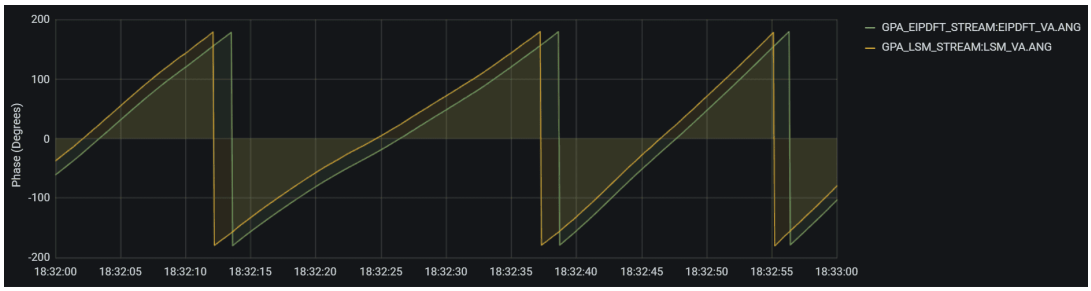


Fig. 11. Real-time phase angle estimations for Phase A (18:32:00 to 18:33:00 UTC). A visible temporal delay of approximately 2 seconds is present between the outputs of the two algorithms.

Fig. 12. Three-phase voltage magnitude tracking (V_A, V_B, V_C) recorded by the e-IpDFT algorithm (18:32:00 to 18:33:00 UTC), demonstrating the operational stability of the C++ wrapper across multiple data channels.

phase errors and the oscillations seen in the dynamic step-tests Figure 8.

The e-IpDFT, however, adapts to the signal. By using a Hanning window to reduce spectral leakage and estimating a fractional bin offset, the e-IpDFT tracks the actual frequency instead of assuming it is exactly 50 Hz. As a result, its TVE stayed well below 0.02%, with the FE and RFE also remaining comfortably below their strict IEEE limits across all steady-state simulations.

B. Real-World Frequency Tracking and Local Variations

Despite failing to meet IEEE limits under both nominal and off-nominal conditions in the MATLAB simulations, the LSM provided comparable frequency measurements to the officially reported Svenska Kraftnät frequency when deployed on the live KTH grid, as seen in Figure 9b.

The reason is that the Nordic power system is highly stable. Under normal conditions, the grid frequency stays very close to 50 Hz. In this narrow range, the LSM's calculation error is small enough to go visually unnoticed on a standard tracking graph. However, P-Class PMUs are best suited for grid emergencies, such as a disconnected generator or a major fault, where the frequency can deviate suddenly from the nominal one. The MATLAB simulations prove that the LSM would become inaccurate during such an event, while the e-IpDFT would remain accurate.

Additionally, small differences were seen between the PMU's local frequency and the Svenska Kraftnät average. This happens because frequency is measured locally. A sudden load change in Stockholm creates a transient wave that the KTH PMU registers immediately, while the national data provides a smoothed average of the entire Nordic synchronous system.

C. Hardware Limitations and Phase Delay

Live Grafana data in Figure 11 showed a 2-second phase delay between the two algorithms. This confirms an issue found by the previous developer group [6]. It is important to note that this is not a mathematical phase error; both algorithms calculate the phase wrapping (the jump from $+180^\circ$ to -180°) correctly.

Instead, this is a hardware and software timing issue. The OpenPMU framework uses Python to fill memory buffers with UDP data. The LSM is written in Python, so it reads this buffer natively. The e-IpDFT uses a C++ wrapper. While passing memory pointers to C++ lowers the CPU usage (see Table III), managing this cross-language handoff likely causes a buffer backlog. The 2-second offset is therefore a data synchronization delay, not an algorithmic failure.

D. Three-Phase Imbalances and Filtering

During the live test, Phase A consistently measured about 1 V lower than Phases B and C (see Figure 12), and an unexpected phase angle offset appeared between phases B and C. A review of the C++ code confirms that all three phases are processed using the exact same loop. Therefore, these anomalies are physical hardware realities rather than software bugs. The 1 V difference is likely caused by minor manufacturing tolerances in the resistors on the analog input board, or an actual unbalanced load on Phase A in the laboratory. Similarly, the unusual phase offsets point to a physical wiring issue on the analog input board.

Furthermore, the e-IpDFT produced a slightly smoother voltage magnitude trace than the LSM. This reflects a fundamental difference between the algorithms. The e-IpDFT utilizes a Hanning window which, combined with its iterative compensation, provides strong rejection of out-of-band ADC hardware noise. Conversely, the rigid pseudo-inverse matrix of the baseline LSM makes it highly sensitive to noise and harmonics, resulting in a more volatile trace. However, when calculating the frequency and ROCOF derivatives, the e-IpDFT remains sensitive to phase angle volatility. This is why the Exponential Moving Average (EMA) filter ($\alpha = 0.05$) was necessary; without it, the high-speed derivative calculations would be too noisy to read.

E. Sources of Error and Limitations

Although the results support the use of e-IpDFT for real-time phasor estimation, several sources of error and uncertainties must be acknowledged:

- **LSM Code Assumptions:** As noted in Section III-C, the pre-compiled source code for the LSM algorithm was not available for direct review. Although the MATLAB model was carefully built based on its standard mathematical definition, the actual embedded version might contain unknown background filters or code optimizations.
- **Translation Risks:** The MATLAB simulations confirmed that the e-IpDFT performs as expected. However, translating this mathematical logic into a combined C++ and Python environment introduces risks. Even with careful memory management, the programming language differences could cause unwanted background effects.
- **Lack of Real-World Fault Data:** The live KTH grid did not experience any severe disturbances or faults during the observation window. Because of this, it is impossible to definitively conclude that the newly compiled e-IpDFT outperforms the LSM during a real grid emergency. The conclusion that e-IpDFT is better for off-nominal tracking is currently based entirely on the simulated MATLAB results, rather than live physical events.

VIII. CONCLUSION

This thesis evaluated and deployed a frequency-domain digital signal processing algorithm to replace a time-domain algorithm in an open-source Phasor Measurement Unit. By running the Least Squares Method (LSM) and the Enhanced Interpolated DFT (e-IpDFT) concurrently on the same hardware, the project directly compared their performance.

The MATLAB simulations showed that the LSM cannot meet IEEE C37.118 P-Class requirements under steady-state conditions. Its fixed 50 Hz design causes large oscillatory errors when the grid's frequency deviates from the nominal one. In contrast, the e-IpDFT uses a Hanning window and a

fractional bin offset to track frequency changes, keeping errors within the IEEE limits.

To deploy the e-IpDFT on the Raspberry Pi, a cross-language architecture was built. The heavy math was written in C++ to bypass Python's speed limitations, which successfully reduced the overall CPU load. Furthermore, the system's networking architecture was successfully hardened using strict firewall whitelisting. This allowed the PMU to securely stream its baseline measurements to a wide-area concentrator at Lund University while keeping experimental data isolated on a local VPN. While the live benchmarking confirmed that the algorithms function correctly under steady-state conditions, it also revealed hardware-specific issues, such as a 2-second buffer delay and analog component tolerances. Because no severe grid faults occurred during the live test, the off-nominal performance relies on simulated data. Overall, the theoretical and simulated results strongly indicate that e-IpDFT is a more robust and compliant algorithm for performing accurate phasor estimations during off-nominal grid conditions than the LSM.

Looking forward, now that the algorithm refinement objective has been successfully met, future work should focus on the other areas proposed in the original project description: device development and data analysis. Because live testing revealed physical constraints, specifically the 2-second phase desynchronization and the analog voltage imbalances, the next logical step is dedicated device development. Upgrading the analog-to-digital expansion board and replacing the Python-based network dispatcher with a faster compiled language would eliminate the buffer latencies and turn the current prototype into a highly robust and field-ready PMU.

Once the physical device is fully optimized, the project can shift toward wide-area data analysis. By deploying multiple upgraded PMUs across different geographic locations, researchers can collect long-term synchronized data to study real-world grid faults. Additionally, integrating a stronger GPS module capable of maintaining microsecond synchronization in indoor environments would improve the device's flexibility for future laboratory testing and calibration.

ACKNOWLEDGMENT

The author would like to thank Valgerður Jónsdóttir for her supervision, and acknowledge the previous developers of the OpenPMU architecture at KTH whose foundational work made this algorithm refinement possible.

REFERENCES

[1] A. Hadavi, M. Tarafdar Hagh, and S. Ghassem Zadeh, "Frequency stability in modern power systems with 100% renewable energy penetration," *Results in Engineering*, vol. 28, p. 108168, 2025.

[2] S. Wu and Y. Li, "State estimation of distribution network considering data compatibility," *Energy and Power Engineering*, vol. 12, pp. 73–83, 2020.

[3] A. G. Phadke and J. S. Thorp, *Synchronized Phasor Measurements and Their Applications*. Boston, MA: Springer, 2008.

[4] D. M. Laverty, D. J. Morrow, R. Best, and P. A. Crossley, "The OpenPMU platform for open-source phasor measurements," *IEEE Transactions on Instrumentation and Measurement*, vol. 62, no. 5, pp. 1117–1124, 2013.

[5] IEC/IEEE, "Measuring relays and protection equipment - part 118-1: Synchrophasor for power systems - measurements," *IEC/IEEE 60255-118-1:2018*, pp. 1–78, 2018.

[6] E. Bergvall, J. Björns, A. Bonetti, and S. Weideskog, "Implementation of open source PMU system," KTH Royal Institute of Technology, Stockholm, Sweden, Internal Project Report, 2023, available upon request.

[7] GeeksforGeeks. (2026, Apr.) Moore-Penrose Pseudoinverse in Mathematics. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/moore-penrose-pseudoinverse-mathematics/

[8] T. Grandke, "Interpolation algorithms for discrete Fourier transforms of weighted signals," *IEEE Transactions on Instrumentation and Measurement*, vol. 32, no. 2, pp. 350–355, 1983.

[9] A. Derviškadić, P. Romano, and M. Paolone, "Iterative-interpolated DFT for synchrophasor estimation: A single algorithm for P- and M-class compliant PMUs," *IEEE Transactions on Instrumentation and Measurement*, vol. 67, no. 3, pp. 547–558, 2018.

[10] H. A. A. Mohammed. (2026, Apr.) OpenPMU e-IpDFT Implementation and Hardware CAD Models. [Online]. Available: https://github.com/AKDontMiss/BSc_Thesis/

[11] Lund University. (2026, May) Nordic PMU Data Network. [Online]. Available: http://skrymer.iea.lth.se/status/

[12] Svenska Kraftnät. (2026, May) The Control Room - National Grid. [Online]. Available: https://www.svk.se/en/national-grid/the-control-room/